

TRAINING PROGRAM • MODULE GUIDE

MLOps & Deployment

A practical, code-first learning resource covering the full path from a trained model to a running, monitored service in production.

Docker

FastAPI

REST APIs

Model Serialization

GitHub Actions

CI/CD

MLflow

AWS & Azure

Final Project

Contents

1. Docker Basics	3
2. FastAPI	5
3. REST API Creation	7
4. ML Model Serialization (pickle / joblib)	9
5. GitHub Actions Overview	11
6. CI/CD Concepts	13
7. MLflow Overview	14
8. Cloud Deployment Overview (AWS / Azure)	16
9. Final Project Presentation Guide	18

HOW TO USE THIS GUIDE

Each section has: the core concept, a working code example, common pitfalls, and a mini-checklist. Read one section per sitting, then rebuild the example from memory — that's what will stick for the viva.

01 Docker Basics

Docker packages your application (code + dependencies + runtime) into a single, portable unit called a **container**. This solves "it works on my machine" by making the environment part of what you ship — critical once an ML model needs to run identically on your laptop, a teammate's machine, and a cloud server.

Core Concepts

Term	Meaning
Image	A read-only template/blueprint (code + OS layer + dependencies)
Container	A running instance of an image
Dockerfile	Instructions to build an image
Registry	Storage for images (Docker Hub, ECR, ACR, GHCR)
Volume	Persistent storage that survives container restarts
docker-compose	Tool to define and run multi-container apps via YAML

A Minimal Dockerfile for an ML/FastAPI Service

```
# Dockerfile
FROM python:3.11-slim

WORKDIR /app

# Install dependencies first (better layer caching)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

WHY COPY REQUIREMENTS.TXT FIRST?

Docker caches each layer. If you copy all code before installing dependencies, every code change invalidates the dependency-install cache and reinstalls everything. Copying `requirements.txt` first means dependencies are only reinstalled when they actually change.

Essential Commands

```
docker build -t my-ml-api .           # build an image from Dockerfile
docker run -p 8000:8000 my-ml-api     # run container, map host:container port
docker run -d --name api my-ml-api    # run detached (background)
docker ps                             # list running containers
docker logs api                       # view container logs
docker exec -it api bash              # open a shell inside the container
docker stop api && docker rm api       # stop and remove
docker images                         # list local images
docker system prune -a                # clean up unused images/containers
```

docker-compose for Multi-Service Apps

Most real ML systems are more than one container — an API, a database, maybe MLflow's tracking server. `docker-compose.yml` lets you define and start all of them together.

```
# docker-compose.yml
version: "3.9"
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - MODEL_PATH=/app/models/model.pkl
    depends_on:
      - db
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

```
docker compose up -d      # start all services in background
docker compose down       # stop and remove containers
docker compose logs -f api # follow logs of one service
```

COMMON PITFALL

Forgetting `.dockerignore`. Without it, your build context includes `venv/`, `__pycache__`, `.git/`, and large datasets — slowing builds and bloating images. Add a `.dockerignore` file just like `.gitignore`.

Docker — Viva-ready checklist

- ☐ Explain image vs. container in one sentence each
- ☐ Write a Dockerfile from scratch for a Python API
- ☐ Explain layer caching and why instruction order matters
- ☐ Know the difference between `CMD` and `ENTRYPOINT`
- ☐ Explain what a Docker volume is and why it's needed for stateful data
- ☐ Be able to explain `docker-compose` and when to use it over a single container

02 FastAPI

FastAPI is a modern Python web framework built for speed and correctness — it uses Python type hints to auto-generate validation, serialization, and interactive API docs (Swagger UI). It's the most common choice for wrapping an ML model behind an HTTP API, and you already have hands-on experience with it in Aether.

Minimal FastAPI App

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import numpy as np

app = FastAPI(title="Model Serving API")

model = joblib.load("model.pkl")

# Pydantic model = request schema + automatic validation
class PredictRequest(BaseModel):
    feature1: float
    feature2: float
    feature3: float

class PredictResponse(BaseModel):
    prediction: float
    confidence: float

@app.get("/health")
def health_check():
    return {"status": "ok"}

@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest):
    x = np.array([[request.feature1, request.feature2, request.feature3]])
    pred = model.predict(x)[0]
    proba = model.predict_proba(x).max()
    return PredictResponse(prediction=float(pred), confidence=float(proba))
```

```
uvicorn main:app --reload --port 8000
# Swagger docs auto-generated at http://localhost:8000/docs
```

Why FastAPI (vs. Flask) for ML Serving

Feature	FastAPI	Flask
---------	---------	-------

Data validation	Automatic via Pydantic	Manual
Interactive docs	Auto-generated (Swagger/ReDoc)	Needs extension
Async support	Native <code>async def</code>	Limited
Performance	High (Starlette + ASGI)	Lower (WSGI, sync)
Type safety	Enforced by type hints	None built-in

Dependency Injection & Path/Query Parameters

```
from fastapi import Depends, Query

def get_model():
    # load once, reuse across requests
    return app.state.model

@app.get("/items/{item_id}")
def read_item(item_id: int, q: str | None = Query(default=None, max_length=50)):
    return {"item_id": item_id, "q": q}

@app.post("/predict")
def predict(request: PredictRequest, model=Depends(get_model)):
    ...
```

Error Handling

```
from fastapi import HTTPException

@app.post("/predict")
def predict(request: PredictRequest):
    if request.feature1 < 0:
        raise HTTPException(status_code=400, detail="feature1 must be non-negative")
    ...
```

LOAD THE MODEL ONCE, NOT PER-REQUEST

Load the model at startup (module scope or an `@app.on_event("startup")` handler), not inside the endpoint function — reloading a pickle file on every request is slow and wastes memory.

FastAPI — Viva-ready checklist

- ☐ Explain Pydantic's role in request validation
- ☐ Difference between path params, query params, and request body
- ☐ What ASGI is and why FastAPI is async-capable
- ☐ How Swagger UI ([/docs](#)) gets generated automatically
- ☐ Where and when to load an ML model in the app lifecycle

03 REST API Creation

REST (Representational State Transfer) is an architectural style for designing networked APIs around **resources** (nouns) manipulated via standard HTTP methods (verbs). Good REST design makes an API predictable and easy to consume.

HTTP Methods & Status Codes

Method	Use	Idempotent?
GET	Retrieve a resource	Yes
POST	Create a resource / trigger an action	No
PUT	Replace a resource entirely	Yes
PATCH	Partially update a resource	No
DELETE	Remove a resource	Yes

Code	Meaning
200 OK	Successful GET/PUT/PATCH
201 Created	Successful POST that created a resource
204 No Content	Successful request, no body to return
400 Bad Request	Client sent invalid data
401 / 403	Unauthenticated / Unauthorized
404 Not Found	Resource doesn't exist
422 Unprocessable Entity	Validation failed (FastAPI default for bad Pydantic input)
500 Internal Server Error	Unhandled server-side exception

Resource-Oriented URL Design

```
# Good – nouns, plural, hierarchical
GET    /models           # list all models
POST   /models           # register a new model
GET    /models/{model_id} # get one model
DELETE /models/{model_id} # delete a model
POST   /models/{model_id}/predict # action on a resource

# Avoid – verbs in the URL
GET /getAllModels
POST /createModel
```

A Small CRUD Example in FastAPI

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()
db: dict[int, dict] = {}
next_id = 1

class ModelInfo(BaseModel):
    name: str
    version: str

@app.post("/models", status_code=201)
def create_model(m: ModelInfo):
    global next_id
    db[next_id] = m.dict()
    next_id += 1
    return {"id": next_id - 1, **m.dict()}

@app.get("/models/{model_id}")
def get_model(model_id: int):
    if model_id not in db:
        raise HTTPException(status_code=404, detail="Model not found")
    return db[model_id]

@app.delete("/models/{model_id}", status_code=204)
def delete_model(model_id: int):
    db.pop(model_id, None)
    return None
```

Versioning & Auth

- **Versioning:** prefix routes with `/v1/`, `/v2/` so breaking changes don't disrupt existing clients.
- **Auth:** API keys via headers, or OAuth2/JWT bearer tokens — FastAPI has built-in `OAuth2PasswordBearer` support for this.

REST API — Viva-ready checklist

- ☐ Explain statelessness — a defining REST constraint
- ☐ Map CRUD operations to HTTP methods correctly
- ☐ Explain idempotency with an example
- ☐ Design a resource-oriented URL scheme for a given feature
- ☐ Know 3–4 common status codes and when each applies

04 ML Model Serialization

Serialization converts a trained model object in memory into a file you can save, move, and reload later — this is what lets a model trained in a notebook be loaded inside a FastAPI service.

pickle vs joblib

Aspect	<code>pickle</code>	<code>joblib</code>
Standard library	Yes (built-in)	No (separate package)
Best for	General Python objects	Objects with large NumPy arrays (sklearn models)
Performance	Slower for large arrays	Faster, more memory-efficient for numeric data
Compression	Manual	Built-in (<code>compress=</code> param)

Saving & Loading with joblib

```
import joblib
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
model.fit(X_train, y_train)

# Save
joblib.dump(model, "model.pkl")
joblib.dump(model, "model_compressed.pkl", compress=3)

# Load
loaded_model = joblib.load("model.pkl")
predictions = loaded_model.predict(X_test)
```

Saving & Loading with pickle

```
import pickle

with open("model.pkl", "wb") as f:
    pickle.dump(model, f)

with open("model.pkl", "rb") as f:
    loaded_model = pickle.load(f)
```

SECURITY NOTE

Never `pickle.load()` a file from an untrusted source — unpickling can execute arbitrary code. Only load pickle/joblib files you or your pipeline produced.

VERSION MISMATCH PITFALL

A model pickled with scikit-learn 1.3 may fail (or silently misbehave) if loaded with scikit-learn 1.5. Always pin the exact library versions in `requirements.txt` and, ideally, store the training environment's version alongside the model file.

Beyond pickle: Framework-Native Formats

Framework	Native format
scikit-learn	joblib / pickle
PyTorch	<code>torch.save(model.state_dict(), "model.pt")</code>
TensorFlow/Keras	<code>model.save("model.h5")</code> or SavedModel format
Cross-framework	ONNX — portable across frameworks and runtimes

Bundling the Model with Metadata

In production it's good practice to save the model together with preprocessing objects and metadata, not just the raw estimator:

```
import joblib
from datetime import datetime

artifact = {
    "model": model,
    "scaler": scaler,          # StandardScaler used during training
    "feature_names": ["f1", "f2", "f3"],
    "trained_at": datetime.utcnow().isoformat(),
    "sklearn_version": "1.5.0",
}
joblib.dump(artifact, "model_bundle.pkl")
```

Serialization — Viva-ready checklist

- ☐ Explain why pickle/joblib alone isn't a full deployment strategy
- ☐ State when to prefer joblib over pickle
- ☐ Explain the version-mismatch risk and how to mitigate it
- ☐ Know at least one framework-native alternative (ONNX, SavedModel)

05 GitHub Actions Overview

GitHub Actions is GitHub's built-in automation platform. You define **workflows** as YAML files in `.github/workflows/` that run automatically on events like a push, pull request, or a schedule.

Key Vocabulary

Term	Meaning
Workflow	An automated process defined in a YAML file
Event	What triggers the workflow (push, pull_request, schedule...)
Job	A set of steps that run on the same runner (VM)
Step	A single command or reusable <code>action</code>
Runner	The VM that executes the job (GitHub-hosted or self-hosted)
Action	A reusable unit of code (e.g. <code>actions/checkout</code>)
Secret	Encrypted environment variable (API keys, credentials)

Example: Test on Every Push

```
# .github/workflows/test.yml
name: Run Tests

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pytest
```

```

- name: Run tests
  run: pytest tests/ -v

- name: Lint
  run: |
    pip install flake8
    flake8 app/ --max-line-length=100

```

Example: Build & Push a Docker Image

```

name: Build and Push Docker Image

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKER_USERNAME }
          password: ${ secrets.DOCKER_PASSWORD }

      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: myuser/my-ml-api:latest

```

SECRETS, NEVER HARDCODED

Credentials go in **Settings** → **Secrets and variables** → **Actions** on GitHub, referenced as `${ secrets.NAME }`. Never commit API keys or passwords into a workflow file.

GitHub Actions — Viva-ready checklist

- ☐ Explain the workflow → job → step hierarchy
- ☐ Name 2-3 common trigger events
- ☐ Explain what a "runner" is
- ☐ Explain how secrets are stored and referenced
- ☐ Write a simple test-on-push workflow from memory

06 CI/CD Concepts

Continuous Integration (CI) means every code change is automatically built and tested as soon as it's pushed, catching bugs early. **Continuous Delivery/Deployment (CD)** means that once code passes CI, it's automatically packaged and released — delivery stops just short of production and waits for a manual approval; deployment goes all the way to production automatically.

The Pipeline, Stage by Stage

Stage	What happens
1. Source	Developer pushes code / opens a PR
2. Build	Install dependencies, compile, build Docker image
3. Test	Unit tests, integration tests, linting
4. Package	Create a versioned artifact (Docker image, wheel, model bundle)
5. Deploy (staging)	Release to a staging environment for validation
6. Deploy (production)	Manual approval (Delivery) or automatic (Deployment)
7. Monitor	Track errors, latency, and — for ML — data/model drift

CI/CD for ML (vs. plain software)

ML pipelines add layers that traditional CI/CD doesn't have to think about:

- **Data validation** — checking schema and data quality before training
- **Model validation** — comparing new model metrics against a baseline before promoting it
- **Model registry** — versioning models the way Git versions code (MLflow Model Registry, for example)
- **Drift monitoring** — the deployed model's real-world accuracy can silently degrade as input data shifts, unlike a static software bug

CT — CONTINUOUS TRAINING

Some teams add a fourth "C": Continuous Training — automatically retraining the model when new data arrives or performance drops, then running it back through the same CI/CD pipeline.

Deployment Strategies

Strategy	How it works
Blue-Green	Two identical environments; switch traffic instantly from old (blue) to new (green)
Canary	Roll out to a small % of traffic first, then increase gradually

Rolling	Replace instances gradually, one/few at a time
Shadow	New model runs alongside the old one on real traffic, but its output isn't served — used to validate before switching

CI/CD — Viva-ready checklist

- ☐ Distinguish Continuous Delivery from Continuous Deployment
- ☐ Walk through the 7 pipeline stages in order
- ☐ Explain what makes ML CI/CD different from software CI/CD
- ☐ Explain canary vs. blue-green deployment

07 MLflow Overview

MLflow is an open-source platform for managing the ML lifecycle: tracking experiments, packaging code into reproducible runs, and managing/deploying models — the piece that turns "which of my 40 notebook runs was the good one?" into a searchable, comparable record.

The Four Components

Component	Purpose
MLflow Tracking	Log parameters, metrics, and artifacts for every run
MLflow Projects	Package code in a reusable, reproducible format
MLflow Models	Standard format to package models for deployment across tools
MLflow Model Registry	Central store for model versions, stage transitions (Staging → Production), and lineage

Tracking an Experiment

```
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

mlflow.set_experiment("customer-churn")

with mlflow.start_run(run_name="rf-baseline"):
    n_estimators = 100
    max_depth = 5

    mlflow.log_param("n_estimators", n_estimators)
    mlflow.log_param("max_depth", max_depth)

    model = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth)
    model.fit(X_train, y_train)
    preds = model.predict(X_test)

    mlflow.log_metric("accuracy", accuracy_score(y_test, preds))
    mlflow.log_metric("f1_score", f1_score(y_test, preds))

    mlflow.sklearn.log_model(model, "model")
```

```
mlflow ui # launches dashboard at http://localhost:5000 to compare runs
```

Registering & Loading a Model

```
# Register the best run's model
mlflow.register_model(
    model_uri="runs:/<run_id>/model",
    name="churn-classifier"
)

# Load a specific version or stage for serving
model = mlflow.sklearn.load_model("models:/churn-classifier/Production")
```

Serving a Model Directly

```
mlflow models serve -m "models:/churn-classifier/Production" -p 1234
# spins up a local REST endpoint for the registered model
```

WHY THIS MATTERS FOR THE VIVA

Be ready to explain the difference between an **experiment** (a group of related runs) and a **run** (one execution with specific params/metrics/artifacts), and how the Model Registry's stage labels (None → Staging → Production → Archived) support safe promotion.

MLflow — Viva-ready checklist

- ☐ Name and describe all four MLflow components
- ☐ Explain `log_param` vs `log_metric` vs `log_artifact`
- ☐ Explain the Model Registry's stage-transition workflow
- ☐ Explain how MLflow supports reproducibility

08 Cloud Deployment Overview

Once a model is containerized, it needs somewhere to run at scale, with reliability and monitoring. AWS and Azure are the two most commonly referenced platforms in interviews — you don't need deep expertise in either, but you should recognize the core services and know what problem each solves.

AWS — Key Services for ML Deployment

Service	Role
ECR (Elastic Container Registry)	Stores Docker images
ECS / Fargate	Runs containers without managing servers
EKS	Managed Kubernetes
Lambda	Serverless functions — good for lightweight/event-driven inference
SageMaker	End-to-end managed ML platform: train, tune, deploy, monitor
S3	Object storage — datasets, model artifacts
CloudWatch	Logging and monitoring

Azure — Key Services for ML Deployment

Service	Role
Azure Container Registry (ACR)	Stores Docker images
Azure Container Instances / App Service	Run containers without managing VMs
Azure Kubernetes Service (AKS)	Managed Kubernetes
Azure Machine Learning	End-to-end managed ML platform, analogous to SageMaker
Blob Storage	Object storage
Azure Monitor	Logging and monitoring

A Typical Deployment Flow

1. Train model → 2. Serialize (joblib) → 3. Wrap in FastAPI
- 4. Dockerize → 5. Push image to registry (ECR/ACR)
- 6. Deploy to managed container service (ECS/AKS/App Service)
- 7. Expose via load balancer / API gateway
- 8. Monitor logs, latency, and prediction drift

Serverless vs. Managed Containers vs. Kubernetes

Option	Best when...
Serverless (Lambda / Azure Functions)	Sporadic traffic, small model, cost-sensitive, low latency tolerance for cold starts
Managed containers (Fargate / App Service)	Steady traffic, don't want to manage a cluster
Kubernetes (EKS / AKS)	Complex multi-service architecture, need fine-grained scaling/orchestration control
Managed ML platform (SageMaker / Azure ML)	Want built-in training, tuning, and deployment pipeline in one place

INTERVIEW FRAMING

If asked "how would you deploy this model," walk through the flow above rather than naming a single service — interviewers are usually testing whether you understand the pipeline, not whether you've memorized a specific vendor's console.

Cloud Deployment — Viva-ready checklist

- ☐ Name the AWS/Azure equivalent for: container registry, container runtime, managed ML platform, object storage
- ☐ Explain serverless vs. managed containers vs. Kubernetes trade-offs
- ☐ Describe the end-to-end deployment flow from trained model to live endpoint

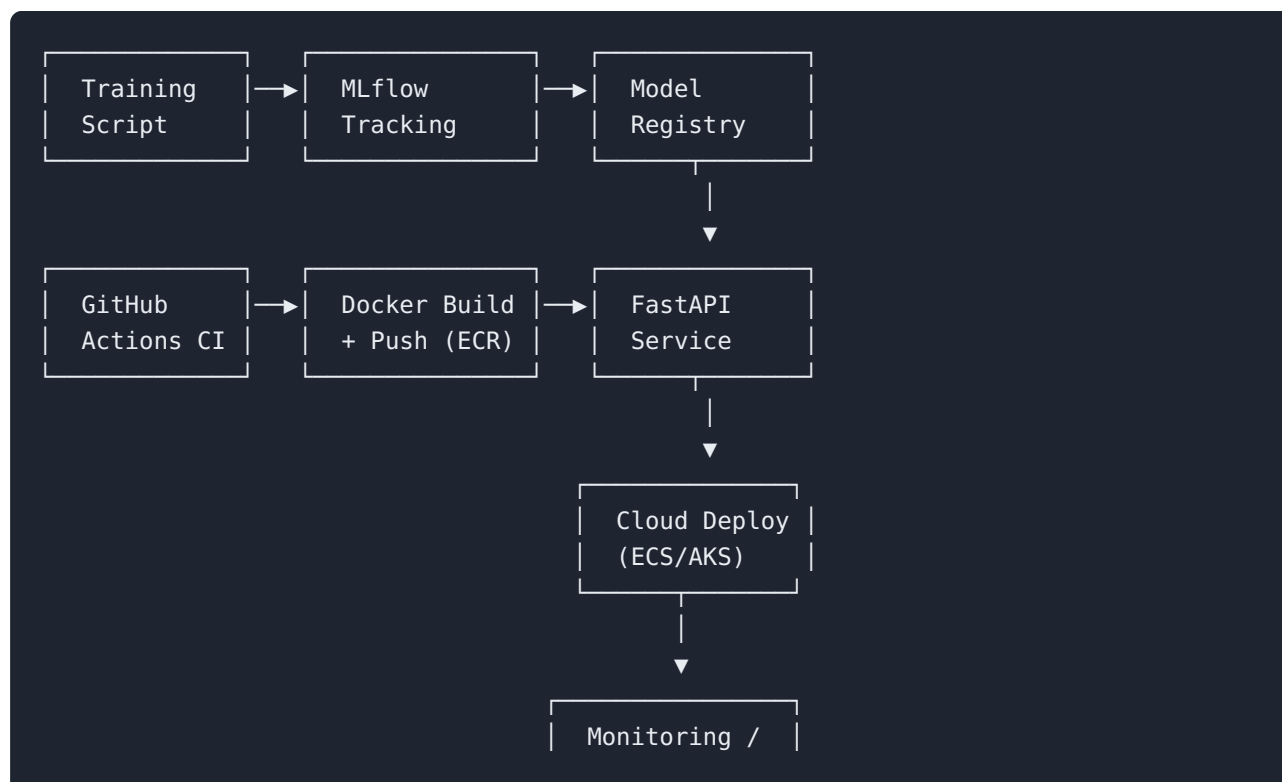
09 Final Project Presentation Guide

This module usually ends with presenting an end-to-end MLOps project. The strongest presentations tell a clear story: problem → data → model → deployment → results → what you'd improve.

Suggested Structure (10-12 minutes)

Section	Time	Content
Problem & Motivation	1 min	What problem, why it matters, who the "user" is
Data & Approach	2 min	Data source, key preprocessing decisions, model choice and why
Architecture	2-3 min	Diagram: training pipeline → serialized model → FastAPI → Docker → CI/CD → cloud
Live/Recorded Demo	2-3 min	Hit the API, show Swagger docs, show a prediction end-to-end
CI/CD & Monitoring	1-2 min	Show the GitHub Actions run, MLflow experiment comparison
Results & Limitations	1 min	Metrics achieved, honest limitations, what you'd do with more time
Q&A	remaining	Anticipate "why this model," "how does it scale," "what if data drifts"

An Architecture Diagram Worth Having Ready



Common Questions Examiners Ask

- "Why did you choose this model over alternatives?"
- "How would you handle 10x the current traffic?"
- "What happens if the input data distribution changes after deployment?"
- "Walk me through what happens from a `git push` to the model being live."
- "How would you roll back a bad deployment?"
- "What's in your Dockerfile, and why is it structured that way?"

PRESENTATION TIP

Have one slide that is just the architecture diagram, and be ready to narrate it end-to-end without notes — this single slide usually carries the most weight in how "production-ready" your project appears.

Final Project — Pre-presentation checklist

- ☐ API is running and reachable — test it minutes before presenting, not the night before
- ☐ Swagger UI (`/docs`) opens cleanly and demo request works
- ☐ Dockerfile builds cleanly from a fresh clone
- ☐ At least one CI workflow run is green in GitHub Actions history
- ☐ MLflow run history has more than one experiment to show comparison
- ☐ You can answer "why" for every architectural choice, not just "what"
- ☐ Backup plan ready in case live demo fails (recorded video / screenshots)